One operator, three decisions, one evidence trail



AI-generated concept illustration; not an installed control room.

Voltage support
Which inverter command meets the limits?

Restoration
Which permitted path can restore service?

Cyber response
Which evidence supports the estimated state?

Learning goal: follow a request through tools, decisions and checks until an operator can explain why to accept, hold or reject it.

U.S. DOE, ADMS Insights (2015). Illustration: `figures/AI_Illustration_Provenance.md`.

A three-hour route from diagrams to operator review

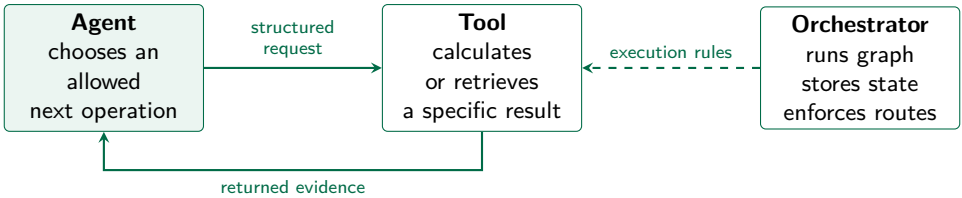
Minutes	Explain	Do
0–40	Graphs, state, decisions and agents	Trace a LangGraph toy; reject stale evidence
40–82	Voltage and MPC/SAC proposals	Compare controls and Llama trials
82–92	**Break**	Keep the notebook kernel ready
92–119	Restoration and discrete choices	Change tie capacity and inspect service
119–149	Cyber response and estimation	Challenge corrupted measurements
149–162	Research and scientific comparison	Compare outcomes, evidence and latency
162–180	Control-room capstone	Explain an acknowledgment or a hold

Each exercise: predict → run → explain the returned evidence.

Labels matter: fresh calculation, saved Llama response and live call are different experiments.

Workshop route: this tutorial. Evidence evaluation: PowerAgentBench-SS (2026 preprint).

Tool, agent and orchestrator have different jobs



Example: Llama proposes a trial; a power-flow tool returns voltages; the orchestrator requires verification before a recommendation.

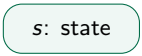
An LLM call can live inside a graph node. A numerical node can run without any LLM. Several roles can use the same Llama backbone.

Yao et al., ReAct (ICLR 2023); LangGraph workflows and agents, accessed 2026-10-17.

Read the diagram: operation, state, dependency and decision



A function that consumes data and returns a result.



The current record: request, command, results and status.

$A \to B$: finish the required result from A before B.

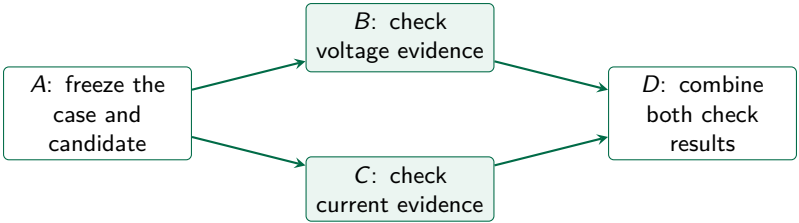


r: routing rule; its returned label selects the next path.

A workflow graph shows **computation dependencies**. Its edges are not the feeder's electrical lines.

LangGraph Graph API; NetworkX topological sort. Tutorial schematics.

A dependency DAG can split work and join the evidence



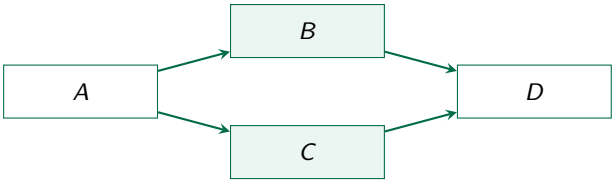
DAG = directed acyclic graph: arrows have direction; no path returns to its starting node.

$G = (N, E)$: $N = \{A, B, C, D\}$ is the node set; E is the set of dependency arrows.

Fan-out: B and C receive the same frozen input. **Join:** D needs both outputs.

NetworkX DAG ordering; LangGraph joins. Conceptual checks; not the G-lab topology.

Topological order puts every prerequisite before its consumer



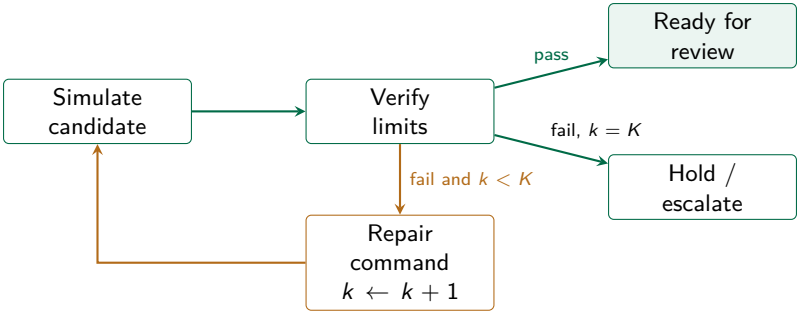
A: freeze input
B: voltage check
C: current check
D: combine

Candidate order	Does it respect all arrows?
A, B, C, D	✓ Yes.
A, C, B, D	✓ Yes; B and C are independent.
A, B, D, C	— No; D is missing C's result.

A schedule can group $\{B, C\}$ together. The join must still wait for both.

NetworkX topological sort. Tutorial's four-node example.

A retry loop is a graph with a cycle, not a DAG



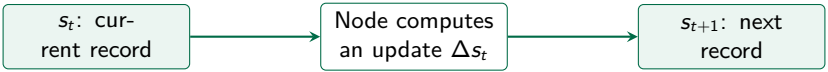
k: completed repairs; K: maximum allowed repairs; both non-negative integers.

Every changed candidate returns to simulation. A finite budget guarantees a stop *if each tool call returns*; it does not guarantee a feasible action.

LangGraph overview; Graph API examples. Retry rule is the tutorial's policy.

Typed state is the record carried through the graph

Field	Type / example	Why the next node needs it
`command`	Number, 0.75	Exact proposed input
`current`, `limit`	Numbers, same declared units	A meaningful comparison
`attempts`, `max_repairs`	Integers, 0 and 2	Remaining retry budget
`passed`	Boolean	Numerical gate result
`trace`	List of event strings	Inspectable execution history



t: workflow step; Δs_t: returned fields, not arithmetic subtraction.

LangGraph state; Python TypedDict. Illustrative G-lab fields.

A reducer states how a field absorbs new information

Replace a current result

Existing `current`	1.00
New `current`	0.80
Stored result	0.80

Only the latest result belongs to the current command.

Append an event to history

`[simulate]` + `[verify]`

`[simulate, verify]`

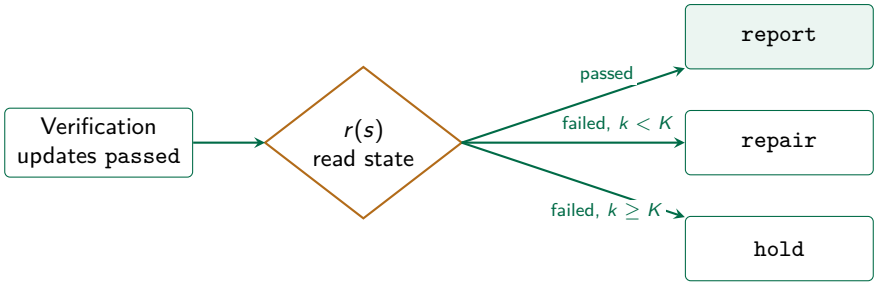
The list-addition reducer preserves both events.

$R_k(a, b)$: reducer for field k, combining stored value a and returned update b.

Design question: if two parallel branches write the same field, what merge rule is valid? Never let accidental write order define the answer.

LangGraph reducer definitions; worked examples. Values are illustrative.

Conditional routing makes the next action explicit



s: state; $r(s)$: one route label; k: repairs used; K: repair budget.

Separate responsibilities: verification computes admissibility; routing decides what the workflow does with that result.

LangGraph conditional-edge examples. Three-way policy: `Graph_Workflow_Lab.ipynb`, G2.

A checkpoint and an interrupt make review resumable

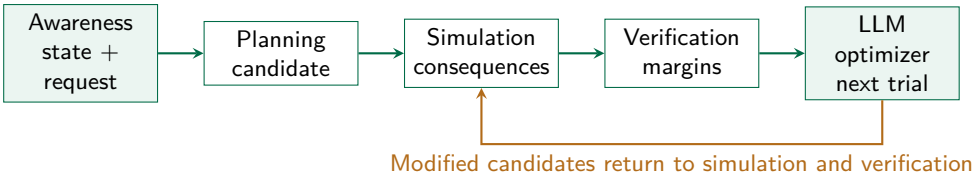


Term	Meaning for this example
Checkpoint	Saved state, associated with a `thread_id`.
Interrupt	Pause that exposes a review request to the caller.
Resume value	Explicit response returned to the paused node.

The review node can run again on resume. Put no uncontrolled equipment side effect before the pause. The toy records acknowledgment only.

LangGraph persistence; interrupt/resume documentation, accessed 2026-10-17.

Five proposed stages share one candidate and its evidence



Shared record: case, state version, command, model scope, limits, tool result and verdict.

Operator handoff: selected action, supported public explanation, alternatives and unresolved assumptions.

An agent is a responsibility; numerical functions can implement much of it. Execution and acknowledgment support the five stages.

SmartGridComm 2026 tutorial proposal, pp. 1–2. Roles: Wu et al., AutoGen (2023 preprint).

Typed, current and feasible are three separate claims

Check	Question	What a pass does not establish
Schema	Are required fields and values valid?	Correct interpretation of operator intent
Binding	Is this result for this request and command?	Authenticity of its source
Engineering gate	Does it meet modeled constraints?	Adequate model or field authorization

Illustrative result: limit 100 A; calculated current 112 A; solver converged.

Decision: REJECT. Two favorable LLM responses cannot change the declared 100 A limit.

PowerDAG (2026 preprint); Grid-Agent (2025 preprint, v3). Current example is illustrative.

Recent frameworks provide mechanisms we can test

Work	Mechanism	Classroom question
PowerAgent / PowerMCP	Engineering-tool interfaces	Can we inspect arguments and units?
PowerDAG	Retrieval and prerequisite guards	Does calculation use current state?
Grid-Agent v3	Sandbox, validation and rollback	Can rejection replace the incumbent?
X-GridAgent	Planning–coordination–action hierarchy	What must survive each handoff?
PowerAgentBench-SS	Matched tools; hidden revalidation	Is the reported claim supported?

Research lesson: compare mechanisms under stated scope. Five roles, tool use and verification already have close prior art.

PowerAgent; PowerDAG; Grid-Agent; X-GridAgent; PowerAgentBench-SS. Review: 2026-10-17.

A capability tick describes scope, not superiority

Work	Tools	Feedback	Checks	Review	Qualification
PowerAgent ecosystem	✓	✓	△	✓	Workflow-specific capabilities
PowerDAG	✓	✓	△	NR	Guards cover selected tools
Grid-Agent	✓	✓	✓	△	Sandbox; proposed oversight
X-GridAgent	✓	✓	△	△	Solver checks; prompt refinement
PowerAgentBench-SS	✓	✓	✓	△	DC benchmark; hidden revalidation
Tutorial workflow	✓	✓	✓	✓	Declared models; local readback

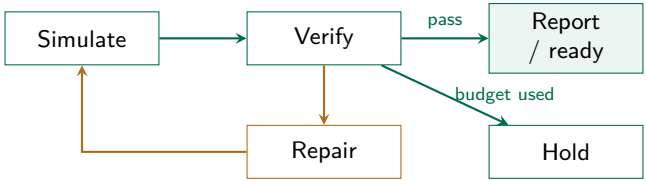
✓ documented △ qualified / proposed NR: not reported in inspected source.

Test the benefit: same tools, scenario and trial budget; compare outcome, evidence quality and latency.

Definitions: `appendix/Power_Frameworks_Comparison.md`. Evaluation: PowerAgentBench-SS. Current versions checked 2026-10-17.

The G lab makes every state transition visible

Notebook G0–G5 `Graph_Workflow_Lab.ipynb`: a real LangGraph toy demonstrator.



Toy response: $i = 0.4 + 0.8u$. u: dimensionless command; i: normalized illustrative current. Pass if $i \leq L$, where L is the declared limit.

Repair: reduce u by 0.25, bounded below by zero. This is *not* a feeder model, MPC or learned policy.

`Graph_Workflow_Lab.ipynb`, G1–G4; LangGraph official documentation.

G1: give the graph a record and an event history

Notebook G1 Inspect the complete state schema and the trace reducer.

```
class TrialState(TypedDict):
    command: float
    limit: float
    attempts: int
    max_repairs: int
    current: float
    passed: bool
    status: str
    acknowledged: bool
    trace: Annotated[list[str], add]
```

Read the code: `TypedDict` names fields; `Annotated` attaches the reducer; `add` appends event lists.

Predict: after simulate and verify both append one event, should `trace` contain one entry or two? **Two.**

Graph_Workflow_Lab.ipynb, G1; Python TypedDict; LangGraph reducers.

G2–G3: bind node functions to permitted routes

Notebook G2–G3 Read the route, then build and compile the graph.

Decision rule

```
def route(s):
    if s["passed"]:
        return "report"
    return "repair" if s["attempts"] < s["
     max_repairs"] else "hold"
```

Graph connections

```
    builder.add_edge(START, "simulate")
    builder.add_edge("simulate", "verify")
    builder.add_conditional_edges("verify",
     route,
        {"report": "report", "repair": "repair",
      "hold": "hold"})
    builder.add_edge("repair", "simulate")
    builder.add_edge("report", END)
    builder.add_edge("hold", END)
    return builder.compile(checkpointer=
     checkpointer)
```

Code to diagram: node names identify work; edges identify allowed successors; the conditional edge reads a route label.

Graph_Workflow_Lab.ipynb, G2–G3; LangGraph construction examples.

G4: predict three paths before running the graph

Trial	Initial u	Limit L	Repairs K	Expected path
Already admissible	0.25	0.80	2	Ready; $i = 0.60$
One repair suffices	0.75	0.80	2	Repair once; ready at $i = 0.80$
Budget exhausted	0.90	0.50	1	Repair once; hold at $i = 0.92$

u: command; L: limit; K: repair budget; $i = 0.4 + 0.8u$: toy normalized current.

Your turn: 2 minutes

Run G4 unchanged. Follow its trace. In a separate scratch cell, invoke the graph with a new limit or repair budget; predict the route first.

Actual labels: `TOY_READY` / `TOY_HOLD`. The direct-Python baseline produces the same three outcomes.

Exact toy and direct-Python comparison: `Graph_Workflow_Lab.ipynb`, G4. Values follow the declared response law and repair rule.

G5: pause, inspect the checkpoint, then resume review

Notebook G5 Use a checkpointer and a real interrupt; record no equipment action.

```
review_app = build_graph(human_review, InMemorySaver())
config = {"configurable": {"thread_id": "student-" + uuid.uuid4().hex}, "recursion_limit": 40}
pause = review_app.invoke(initial_state(0.75, 0.80, 2), config)
snapshot = review_app.get_state(config)
print("Pending node:", snapshot.next, "| acknowledged:", snapshot.values["acknowledged"])
print("Interrupt payload:", pause["__interrupt__"][0].value)
assert snapshot.next == ("report",) and not snapshot.values["acknowledged"]
response = {"approve": True, "recommendation_id": recommendation_id(snapshot.values)}
resumed = review_app.invoke(Command(resume=response), config)
```

Before resume: pending node `report`; acknowledgment is false.

After the scripted readback: inspect `TOY_ACKNOWLEDGED`; retain the same `thread_id`.

Try a rejection: it must remain distinguishable from approval. This toy does not authenticate an operator.

Graph_Workflow_Lab.ipynb, G5; LangGraph interrupts; persistence.

L1: a changed request needs fresh evidence

Notebook L0–L1 Open `Classroom_Labs.ipynb`; run setup, then the binding experiment.

```
request = lab.validate_request(dict(task='voltage', backend='lindistflow3',
    load_scale=1.0, rating_fraction=0.80, revision=1))
candidate = {'control': [0.25, 0.0]}
evidence = lab.bind_evidence(request, candidate, {'example_only': True, 'passed': True})
```

Compare the saved record with	Expected binding result
Same request and command	True
Changed load with old evidence	False
Changed command with old evidence	False

Explain: hashes bind request, candidate and result. The payload is `example_only`; matching records do not prove physics or source authenticity.

`Classroom_Labs.ipynb`, L1; PowerDAG (2026 preprint).

Model-based planning and model-free learning supply proposals

Model-based: optimize with a model

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \ell(x_k, u_k)$$

subject to predicted behavior and operating limits.

H: horizon; k: predicted step; x_k: state; u_k: action; ℓ: declared cost.

Voltage example: MPC plans several actions and uses the first.

Model-free: learn an action rule

$$u_t = \pi_\theta(o_t)$$

from interactions and rewards during training.

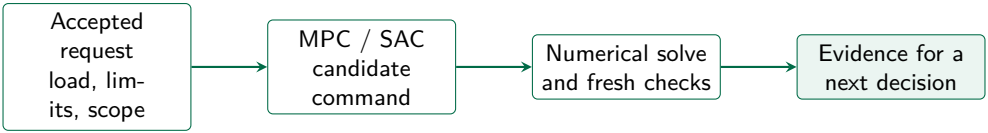
t: decision time; o_t: observation; π_θ: policy with learned parameters θ.

Voltage example: SAC produces continuous controls from saved weights. PPO is another option.

Shared workflow: simulate both proposals and check the same limits. Llama may select permitted follow-up work.

Rawlings, Mayne and Diehl, MPC (2017); Haarnoja et al., SAC (2018); Schulman et al., PPO (2017).

From graph mechanics to a physical voltage decision



Keep fixed: scenario, candidate budget, electrical constraints and objective.

Measure separately: numerical quality, failed checks, Llama's contribution and added time.

The next question is physical: what do active and reactive powers do to feeder voltage and current?

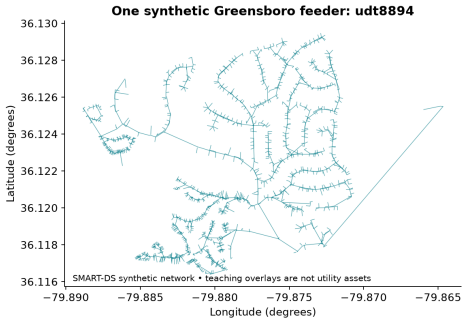
Matched budgets: PowerAgentBench-SS (2026 preprint). Numerical experiment: `Classroom_Labs.ipynb`, L2–L5.

Voltage support: choose the next action, then test its consequences

Operator request
Keep the selected feeder within its modeled limits while using available PV.

Decisions
How much active power? How much reactive support? Which model should judge the proposal?

This chapter's experiment
MPC and SAC propose controls. Language models choose a bounded refinement. A fresh numerical check decides admissibility.

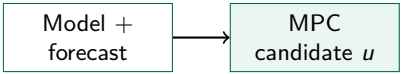


One SMART-DS synthetic Greensboro feeder: `uhs20_1247-udt8894`. The entire city is not modeled.

Palmintier et al., SMART-DS dataset (2020); `data/DATA_SOURCE.md`.

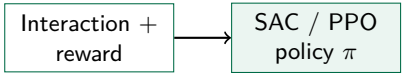
Model-based and model-free methods answer the same action question

Model-based: predict, then optimize



Optimization occurs at decision time. A wrong forecast or missing constraint can invalidate the proposal.

Model-free: learn, then infer



Training occurs before use. A new observation produces an action quickly; distribution shift can make it poor.

u: proposed control; π: learned observation-to-action rule. SAC is executed here; PPO is a discussed continuous-action alternative.

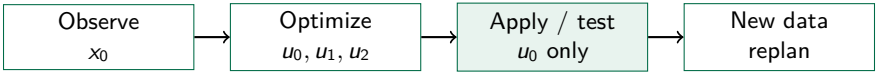
Fair comparison: same scenario, objective and acceptance limits. Both proposals return to a fresh power-flow check; an LLM response does not replace that check.

Rawlings, Mayne & Diehl, Model Predictive Control (2017). Haarnoja et al., SAC, ICML 2018; Schulman et al., PPO (2017).

MPC plans several moves and applies only the first

Read the symbols first: x_k is state, u_k is control and d_k is forecast demand at step k. H is the look-ahead horizon; ℓ scores one step; $\gamma = 0.98$ discounts later steps.

$$\min_{u_0,\dots,u_{H-1}} \underbrace{\sum_{k=0}^{H-1} \gamma^k \ell(x_k, u_k)}_{\text{cost of the proposed sequence}} \quad \text{s.t.} \quad \underbrace{x_{k+1} = f(x_k, u_k, d_k)}_{\text{prediction model}}, \quad \underbrace{g(x_k, u_k) \le 0}_{\text{allowed operating region}} .$$



Our implementation: $H = 3$ quasi-static power-flow snapshots linked by control-movement cost. The generic dynamic equation explains MPC; the tutorial uses algebraic snapshots and does not demonstrate transient stability or a closed-loop rollout.

Rawlings, Mayne & Diehl, Model Predictive Control (2017).

Make the objective interpretable before optimizing it

One two-number fleet command: $u = (a, b)$. For inverter i, S_i is its kVA rating, a is active participation and signed b is reactive participation; both are dimensionless. The one-step teaching cost is ℓ.

$$p_i = aS_i, \qquad q_i = bS_i, \qquad b < 0 \;\Rightarrow\; \text{reactive absorption.}$$

$$\ell = \underbrace{(1-a)^2}_{\text{unused active capability}} + \underbrace{0.05b^2}_{\text{reactive effort}} + \underbrace{10\ \text{mean}[(V-1.01)^2]}_{\text{voltage deviation}} + \underbrace{0.05\|u-u_{\text{prev}}\|_2^2}_{\text{command movement}}.$$

V: modeled voltage magnitudes (p.u.); 1.01: reference (p.u.); u_{prev}: previous command. Model scope determines which voltages enter the mean.

Predict: If we reduce $|b|$, which term certainly decreases? Does total cost necessarily decrease?

Only the reactive-effort term certainly decreases. Voltage, movement cost and feasibility can change too. These weights are teaching preferences under the declared availability assumption. A smaller one-step ℓ does not establish stability or field performance.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`.

A low objective is useful only inside the admissible set

Symbols before limits: V_i is bus-phase voltage; I_e is branch current; lower/upper bounds define the contract. a, b are the fleet command; S_i is inverter rating.

Network admissibility

$$V_{\min} \le |V_i| \le V_{\max}, \quad |I_e| \le I_e^{\max}.$$

Device admissibility

$$a^2 + b^2 \le 1, \quad 0 \le a \le 1, \quad b_{\min} \le b \le b_{\max}.$$

The circle follows from $p_i^2 + q_i^2 \le S_i^2$.

Evidence admissibility
Finite inputs and solver completion; dispatch tracking; matching case, control, model and evidence IDs.

The planner uses local sensitivities and a conservative capability polygon. The proposed action then returns to the selected numerical model.

Predict: Can a majority vote make an excessive current acceptable?

No. Voting selects a trial. The engineering gate determines whether it can be retained.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`. Rawlings, Mayne & Diehl, Model Predictive Control (2017).

LinDistFlow3 and OpenDSS expose different parts of this feeder

	Python LinDistFlow3	**OpenDSS**
Relation	Linear three-phase approximation.	Nonlinear unbalanced power flow.
Network	MV; ideal aggregated LV demand.	Supplied MV/LV and transformers.
Meaning of PASS	Within the reduced contract.	Within the full-feeder snapshot contract.

Decode the voltage-drop model: $\mathbf{w}_i$ is squared phase voltage at parent bus i (V^2); j is its child; Z_{ij} is coupled impedance (Ω); $\mathbf{S}_{ij}$ is downstream phase power (VA).

$$\mathbf{w}_j = \underbrace{\mathbf{w}_i}_{\text{upstream voltage}} \underbrace{- 2\,\mathrm{Re}\left[\overline{\beta} \odot \left(Z_{ij}(\beta \odot \overline{\mathbf{S}}_{ij})\right)\right]}_{\text{three-phase coupled voltage drop}}.$$

$\beta = (1, e^{-j2\pi/3}, e^{-j4\pi/3})$ gives nominal phase ratios; here j in the exponential is the imaginary unit. Bars mean conjugation; $\odot$ means entrywise multiplication; Re takes the real part.

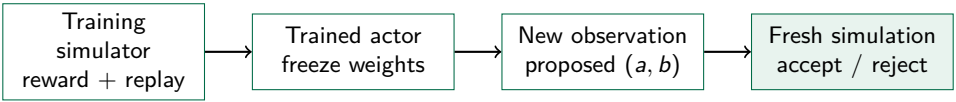
Comparison boundary: different network coverage also changes which constraints can fail. This is not an accuracy-only comparison of equivalent models.

Gan & Low, three-phase distribution optimization (2014); EPRI OpenDSS documentation; `src/lindistflow3.py`.

SAC learns a continuous proposal; verification remains a separate step

Learning vocabulary: s_t is an observation, u_t an action, r a reward and π the actor. $\mathcal{H}$ measures policy entropy; α weights exploration and γ discounts later rewards.

$$\max_{\pi} \mathbb{E}\left[\sum_{t\geq 0} \gamma^t \left\{ \underbrace{r(s_t, u_t)}_{\text{task reward}} + \underbrace{\alpha\, \mathcal{H}(\pi(\cdot \mid s_t))}_{\text{exploration incentive}} \right\}\right].$$



Executed artifact: 12,000 training transitions on a cubic surrogate fitted to 240 AC solves; 48 separate fit-validation samples. New scenarios change current-rating conditions: this is transfer, not retraining. A reward penalty does not prove constraint satisfaction.

Haarnoja et al., SAC, ICML 2018; Schulman et al., PPO (2017). Trained artifact: `Continuous_SAC_Training.ipynb`.

Lab 2: change demand and compare the same two numerical actions

Notebook L2 Fresh MPC solve, saved-actor inference and fresh numerical checks.

```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
started = time.perf_counter()
voltage = lab.voltage_lab(LOAD_SCALE, RATING_FRACTION)
timings['L2 fresh MPC + SAC + checks'] = time.perf_counter() - started
```

Your turn: 7 minutes

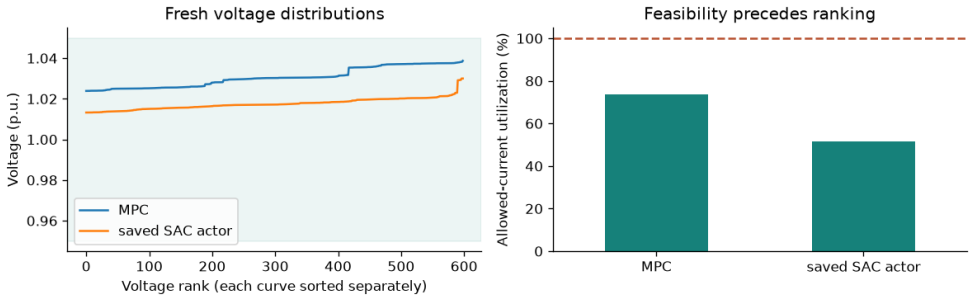
Before running, predict whether a demand change increases PV curtailment or reduces it. Run the default experiment, then change one load multiplier. Compare objective, voltage/current margins and the gate result.

Record three things: the assumption you changed; the numerical response; whether the original recommendation still has matching evidence.

Do not infer: an action that was good at the original load is still good after the change.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Rawlings, Mayne & Diehl, Model Predictive Control (2017).

Read the margins before deciding which controller was better



Compare in this order: did the action pass the same gate? What objective did it achieve? What changed when the demand changed? What was excluded by the model?

Bridge: the numerical methods produced evidence. Now the LLM may choose a bounded next trial from that evidence.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

Actual Llama responses can be useful and still contain unsupported claims

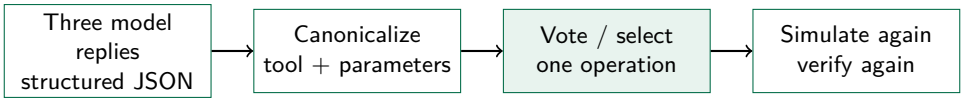
Model / time	Selected trial	Excerpt from its recorded public response
Llama 3.1 8B 1.867 s	Increase a by 0.0025	"Increasing active participation by 0.0025 may help reduce curtailment and improve the overall performance."
Llama 3.2 3B 0.905 s	Reduce $\|b\|$ by 10%	"Reducing reactive effort magnitude by ten percent may help improve the overall system performance and reduce curtailment"
Qwen 2.5 7B 1.844 s	Reduce $\|b\|$ by 10%	"may help improve voltage stability without significantly degrading active power performance"

Notebook L3 Original archived test 01; separate from the changed L2 classroom case.

Critique: less reactive effort alone does not establish less curtailment. These steady-state tests do not establish voltage stability.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`.

Voting selects one physical operation, not the most persuasive paragraph



Two descriptions of the same (a, b) should not become two different actions. Canonical IDs make duplicates and ties reproducible.

Test 01: two models propose the same Q reduction; one proposes a PV increase. The majority selects the Q trial.

Why disagreement matters: it suggests another candidate to test. It does not quantify the probability that the grid action is safe.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`. Ensemble evaluation context: PowerAgentBench-SS (2026).

The LLM optimizer can learn which proposed operation to test

Select → observe → update. Here s is the evidence summary and a is a canonical proposed operation. $\phi(s, a)$ encodes their features; θ stores learned reward coefficients; A stores regularized feature information; β weights exploration. Score q chooses the next operation a^*.

$$q(a \mid s) = \underbrace{\theta^\top \phi(s,a)}_{\text{predicted reward}} + \underbrace{\beta\sqrt{\phi(s,a)^\top A^{-1}\phi(s,a)}}_{\text{training exploration bonus}}, \qquad a^* = \arg\max_a q(a \mid s).$$

After selection, observe reward r and update A and reward vector b:

$$A \leftarrow A + \phi\phi^\top, \qquad b \leftarrow b + r\phi, \qquad \theta = A^{-1}b.$$

This reward vector b is unrelated to inverter b. Frozen L4 evaluation uses predicted reward only, without updates.

Notebook L4 Load the frozen LinUCB selector, choose an operation and recheck it.

Boundary: the bandit selects among already generated responses. It neither selects which LLM to call nor trains LLM weights.

Li et al., LinUCB, WWW 2010; Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`.

Reward learning and safety checking have different jobs

J_{inc} and J_{trial} are incumbent and trial current-step costs; r is feedback after a numerical trial.

$$r = \begin{cases} \min(1, \max(0, J_{\text{inc}} - J_{\text{trial}})/0.01) - 0.01, & \text{trial passes,} \\ -1.01, & \text{trial rejected.} \end{cases}$$

The penalty 0.01 is a fixed trial charge, not latency in seconds.

Learning design: 24 training contexts, 8 validation, 12 test; five training orders over the same data. Frozen validation/test: zero updates.

Your turn: 3 minutes

Run L4. Compare the selected trial with the majority vote. If the reward predicts improvement but fresh verification fails, which result should control the recommendation?

The gate. Retain a valid incumbent or escalate. Reward optimization does not create an operating permission.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`. Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

A rigorous result can show that the extra agent adds no benefit

MV test, 12 scenarios	Mean J	Full-feeder AC PASS	Accounted time
Numerical fixed refinement	0.03400656	0/12	3.327 s
Majority vote	0.03398479	0/12	Pool cost applies
Learned selector	0.03398479	0/12	8.119 s

Learned selection and voting tie on every held-out case. A posthoc fixed Q-reduction rule also reproduces these decisions without LLM calls.

What is demonstrated? Reproducible proposals, traceable refinement and explicit model-scope checking.

What is not demonstrated? Superior control from extra LLMs or learning. A simpler supervisor is a serious comparator.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`.

Lab 5: ask a broader model to audit the recommendation

Notebook L5 Fresh MV check; optional fresh AC audit and separate AC replanning.

```
started = time.perf_counter()
scope_audit = lab.backend_audit(pool, run_opendss=RUN_OPENDSS)
timings['L5 selected model scope checks'] = time.perf_counter() - started
display(scope_audit['table'])
print(scope_audit['optional_mode'])
```

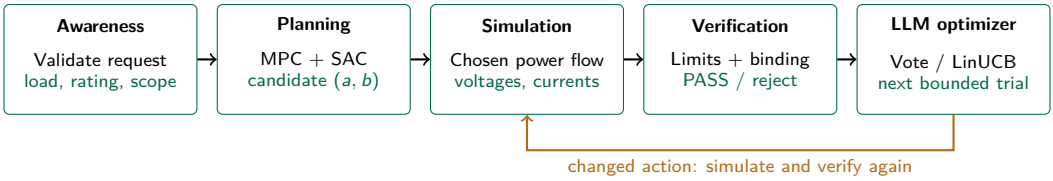
Separate recorded pilot: 12 paired cases

Final learned-workflow result	MV planning	OpenDSS planning
Selected-model final checks	12/12	12/12
Full-feeder AC checks	0/12	12/12
Human operating approval	Pending	Pending

Causal explanation: the network coverage and acceptance contract changed. The learned selector remained frozen. Do not attribute the broader-model benefit to Llama.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`. Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

Voltage support: what each stage contributed



Evidence provenance: L2 performs fresh numerical solves and saved-actor inference. L3–L5 separately inspect the recorded response-selection pilot; its Llama replies are archived.

Operator handoff: retain the checked action, model scope and failed limits. In the recorded test, voting and learned selection tie; the reduced-model actions still fail the broader AC audit. The architecture exposes this boundary rather than proving better control.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`.

Ten-minute break: 82–92 minutes

Keep the notebook kernel running.

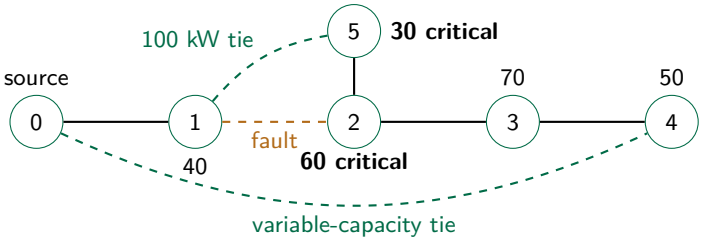
We have adjusted controls on a fixed network.

Next: a fault changes the available paths.

Which loads should the remaining network support?

Workshop timing: ten full minutes; no required work during the break.

After a fault, reconnecting service is a discrete decision



Loads and tie capacities are in kW. This transparent six-node teaching graph is separate from the Greensboro feeder.

Predict: Can bus 4 carry power onward while its own load remains disconnected?

Yes. Energizing a bus and picking up its load are different decisions.

Chen et al., IEEE TPWRS (2018); Wang et al., radiality constraints (2020).

Restoration MILP: choose service while preserving a source path

Three binary choices: s_i picks up load i; y_i energizes its bus; z_e closes edge e. Demand d_i is in kW; priority $w_i = 10$ for critical loads and 1 otherwise. Source $y_0 = 1$.

$$\underbrace{\max \sum_i w_i d_i s_i}_{\text{weighted service}}, \qquad \underbrace{s_i \le y_i, \quad z_{ij} \le y_i, y_j}_{\text{pickup and edges need energized buses}}, \quad z_{\text{fault}} = 0.$$

P_e is signed active flow; $\overline{P}_e$ is edge capacity (kW); A is the node-edge incidence matrix.

$$\underbrace{\sum_e A_{ie} P_e = d_i s_i \quad (i \neq 0)}_{\text{power reaches each served load}}, \qquad \underbrace{|P_e| \le \overline{P}_e z_e}_{\text{open edge carries no flow}}.$$

f_e is a fictitious connectivity commodity, not electricity; each energized non-source bus consumes one unit.

$$\sum_e A_{ie} f_e = y_i \ (i \neq 0), \qquad |f_e| \le 5 z_e, \qquad \underbrace{\sum_e z_e = \sum_i y_i - 1}_{\text{connectivity + edge count gives a tree}}.$$

Scope: six-node active-power transport model; AC voltage, reactive power and switching chronology are omitted.

Chen et al., IEEE TPWRS (2018); Wang et al., radiality constraints (2020).

A fair baseline must optimize the same feasible network

	Unweighted service baseline	Priority-weighted service
Objective	$\max \sum_i d_i s_i$	$\max \sum_i w_i d_i s_i$
Same constraints?	✓	✓
Same fault / capacities?	✓	✓
Exact solution method	Exhaustive enumeration	HiGHS MILP + enumeration check
Report	Total and critical kW	Total and critical kW

Interpretation: a difference caused by priority weights is an optimization-policy effect. It is not evidence that an LLM discovered a better optimizer.

Agent opportunity: turn a documented priority instruction into a validated parameter change; preserve the fault exclusions, action permissions and evidence chain.

Chen et al., IEEE TPWRS (2018); Wang et al., radiality constraints (2020). Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

Lab 6: predict the effect of reducing tie capacity

Notebook L6 Fresh capacity sweep; independent enumeration checks the MILP.

```
CAPACITIES_KW = (40., 60., 80., 100., 120.)
started = time.perf_counter()
restoration = lab.restoration_sweep(CAPACITIES_KW)
timings['L6 fresh MILP sweep + independent enumeration'] = time.perf_counter() - started
display(restoration['table'].drop(columns='pickup'))
```

Your turn: 8 minutes

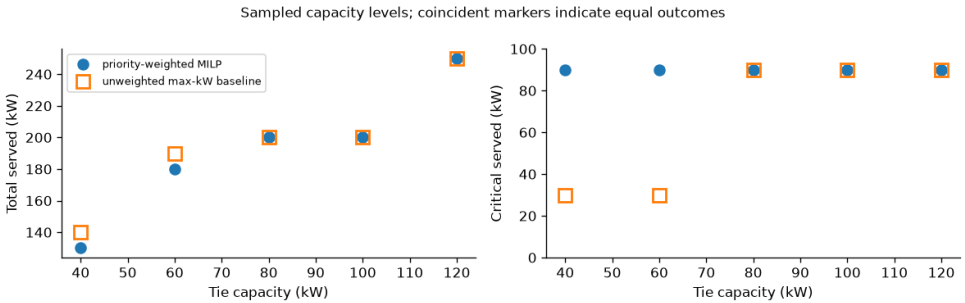
Predict total and critical service at 40, 60, 80 and 120 kW. Run the sweep; add a tighter rating. Compare weighted and unweighted solutions. Explain a flat region in the service curve.

Inspect: picked-up loads, closed edges, critical kW, total kW and enumeration agreement.

Why a sweep? One attractive topology cannot show how priorities and constraints interact.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Chen et al., IEEE TPWRS (2018); Wang et al., radiality constraints (2020).

Discrete service can stay constant while a rating changes



Explain the sampled service levels: loads are indivisible in this model. A tighter rating changes the solution only when it excludes a useful feasible pickup pattern.

At 100 and 80 kW in the priority formulation: 200 kW total, 90 kW critical. Bus 4 may remain energized while its own load is disconnected.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

Lab 7: a changed topology needs a new evidence record

Notebook L7 Try to reuse a result after changing the case.

```
new_rest_request = {**rest_request, 'capacity_kw': 20., 'revision': 2}
print('Original contract:', lab.bound_to(rest_request, rest_candidate, rest_evidence))
print('Changed rating with old evidence:', lab.bound_to(new_rest_request, rest_candidate,
    rest_evidence))
print('Required response: HOLD old recommendation; request a fresh constrained solve.')
```

Your turn: 3 minutes

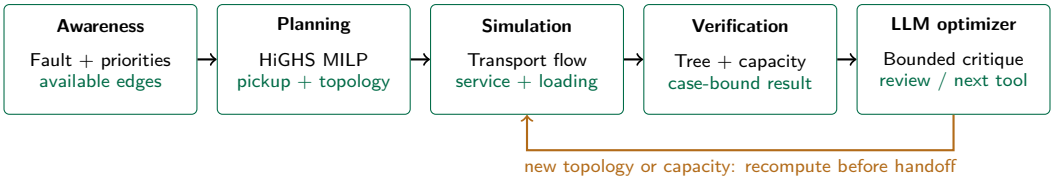
Change a capacity or pickup request while retaining the old evidence. Predict the verifier outcome. Recompute the evidence and repeat the handoff.

The safeguard: a passing result is bound to the case, control, model and contract that were actually checked.

The limit: this cannot establish field isolation, switch availability or protection coordination. Those need separate authorized evidence.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. State-prerequisite guards: PowerDAG (2026).

Use every workflow stage, but attribute the benefit correctly



Evidence provenance: L6–L7 freshly solve the six-node graph and test stale-evidence rejection. The five-stage Llama example is a separate recorded feeder catalog, inspected in L10.

Operator handoff: review isolation, permissions and model scope. In the recorded feeder comparison, the fixed and Llama routes obtain the same service objective; explanation adds traceability and latency, not demonstrated extra restored load.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Actual recorded five-stage reference: `Proposal_Complete_Workflow.ipynb`, P1–P8; `results/proposal_complete_workflow.json`.

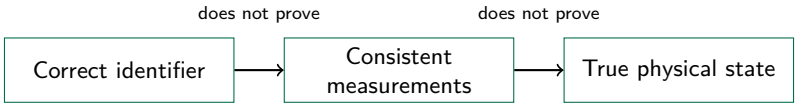
From restoration to cyber response: what if the evidence is wrong?

Established so far

We can optimize a permitted topology and reject evidence that belongs to the wrong case.

A harder question

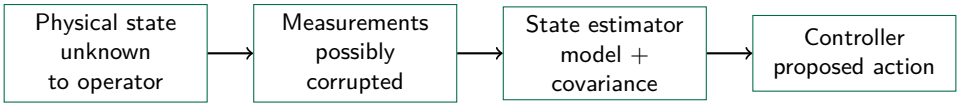
What if the record matches the case, but the measurements inside it are corrupted?



Next experiment: construct a measurement error that the usual residual test cannot see, then inspect its control consequence.

Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

Cyber response begins with an estimation problem



Four-bus DC teaching system: three unknown non-reference voltage angles; eight measurements of flows, injections and one angle.

Two cases: an isolated biased sensor and a coordinated model-consistent perturbation.

The experimenter knows synthetic truth to score the outcome. The agent's operating gate does not receive that truth.

Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

WLS measures consistency with a model and noise assumption

Symbols first: $x \in \mathbb{R}^3$ contains unknown angles (rad); $z \in \mathbb{R}^8$ contains sensor readings; H maps state to measurements. Noise ϵ has covariance R, which weights each sensor in its own units. WLS returns estimate $\hat{x}$.

$$z = Hx + \epsilon, \quad \epsilon \sim \mathcal{N}(0, R), \qquad \hat{x} = \arg\min_x \underbrace{(z - Hx)^\top R^{-1} (z - Hx)}_{\text{weighted mismatch between readings and model}}.$$

The residual r is the unexplained measurement mismatch; J is its dimensionless weighted energy.

$$r = z - H\hat{x}, \qquad J = r^\top R^{-1} r, \qquad \underbrace{J > \chi^2_{5,0.99} = 15.0863}_{\text{five residual degrees of freedom: } 8-3} \Rightarrow \text{alarm}.$$

What this means: readings inconsistent with the assumed model and noise. It does not identify malicious intent.

The 1% nominal false-alarm level assumes a correct linear model, Gaussian noise and full observability. It is not a measured field false-alarm rate.

Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

A coordinated perturbation can change the estimate without the residual

Define c as a state displacement, a as a measurement perturbation and primes as changed quantities. Choose a coordinated pattern $a = Hc$:

$$z' = z + Hc.$$

For the same full-rank linear WLS model, $\hat{x}' = \hat{x} + c$. Therefore

$$r' = z' - H\hat{x}' = z + Hc - H(\hat{x} + c) = z - H\hat{x} = r.$$

Same residual, different estimated state.

A controller can confidently choose an action for the wrong operating point. More eloquent interpretation of the same residual does not recover the missing information.

This construction assumes the attacker can alter the required measurement pattern and knows the relevant linear model. Real restrictions can limit that capability.

Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

Lab 8: sweep the attack size and watch two different signals

Notebook L8 Fresh WLS experiment for isolated and model-consistent perturbations.

```
ATTACK_SCALES = (0., .25, .5, 1., 1.5)
started = time.perf_counter()
cyber = lab.cyber_sweep(ATTACK_SCALES)
timings['L8 fresh WLS attack sweep'] = time.perf_counter() - started
display(cyber['table'])
```

Your turn: 7 minutes

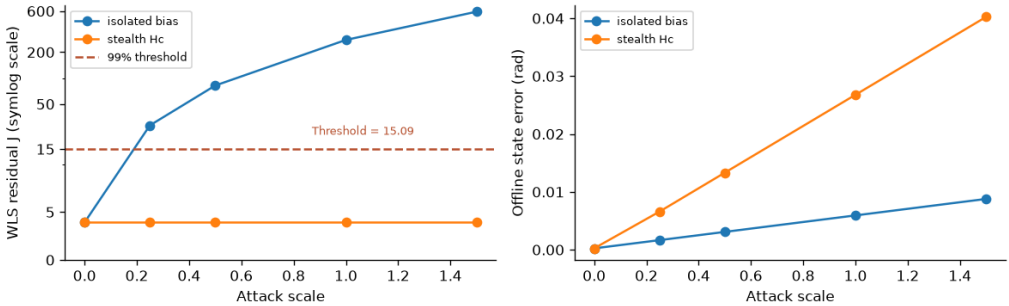
Predict the residual and state-error curves for both perturbations. Run the sweep. Find an example where the residual stays below threshold while state error grows.

Keep two plots separate: a detector sees residual statistics; the experimenter evaluates estimation error using synthetic truth.

Question: what can the agent truthfully conclude from "no residual alarm"?

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

A missed alarm can coexist with a large estimation error



Read left to right: residual consistency and state accuracy answer different questions. A model-consistent perturbation preserves the former while degrading the latter.

Next requirement: an independent, justified information source or a safe escalation path.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

Protected measurements enable a constrained recovery action

Information comes first: $\hat{x}_{\mathcal{T}}$ is the state estimated from three explicitly protected angle readings. The fixture assumes that trust; neither WLS nor the LLM establishes it.

Action and physics: u is added node-2 injection, balanced at the source (p.u.); F maps angles to branch flows; b_u is branch-flow sensitivity to u.

$$\min_{0 \le u \le 0.5} \underbrace{\tfrac{1}{2}u^2}_{\text{smallest corrective injection}} \quad \text{s.t.} \quad \underbrace{|F\hat{x}_{\mathcal{T}} + b_u u|}_{\text{predicted branch-flow magnitudes}} \le \underbrace{\overline{f} - 0.01}_{\text{limits minus planning margin}} .$$

$\overline{f} = (0.35, 0.18, 0.35, 0.35)$ p.u. gives each branch's limit. The 0.01 p.u. margin is an engineering choice, not a calibrated chance constraint.

Predict: Do three independent readings for three unknown states leave a residual consistency test?

No redundancy remains. Rank 3 enables estimation, but residual degrees of freedom are zero. If the protected evidence is unavailable, preserve uncertainty and escalate.

Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009. Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

Lab 9: verify the control consequence under the same synthetic truth

Notebook L9 Compare corrupted-state and protected-state control.

```
without_trust = lab.trusted_response(cyber, protected_available=False)
with_trust = lab.trusted_response(cyber, protected_available=True)
```

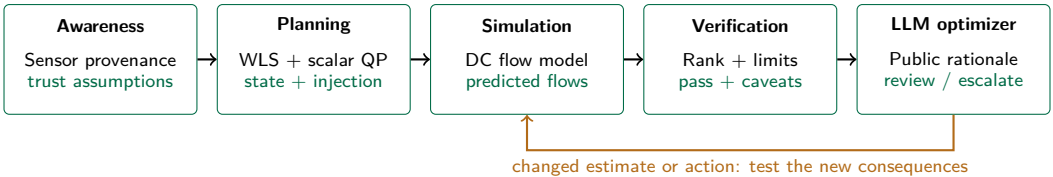
Original toy fixture	u (p.u.)	Actual flow 1–2 (p.u.)
Stealth-corrupted estimate	0	0.24000
Protected estimate	0.12169	0.17046

Your turn: 5 minutes

Run L9. Check the 0.18 p.u. limit and all other branches. Explain why protected information, rather than an LLM label, changes this result.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

The cyber workflow must preserve uncertainty at each handoff



Evidence provenance: L8–L9 freshly evaluate the DC fixture; synthetic truth is for scoring, not given to the operating gate. L10 reads the separate archived five-stage Llama records.

Operator handoff: retain the unverified trust assumptions. The recorded fixed and Llama routes use the same numerical control; improved attack detection is not established by the language layer.

Actual recorded five-stage reference: `Proposal_Complete_Workflow.ipynb`, P1–P8; `results/proposal_complete_workflow.json`. Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

The three cases teach three different failure boundaries

Case	What a naive workflow can miss	Evidence that addresses it
Voltage	A reduced model omits a binding limit.	Broader-model validation and explicit scope.
Restoration	A good plan is applied after topology changes.	Fresh case-bound simulation and permissions.
Cyber	Plausible measurements describe the wrong state.	Justified trusted information or escalation.

Common architecture: choose tools, retain provenance, enforce contracts, expose limitations and involve the operator.

Research question: when does learning improve this architecture enough to justify its additional time, cost and failure modes?

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`. Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

Recent research asks when coordination is worth its cost

Research result: Kim et al. study 260 configurations across six agent benchmarks. Collaboration depends on task structure and the capability of the single-agent baseline.

Design choice	Possible benefit	What our experiment must measure
Parallel specialists	Explore different candidates	Distinct valid actions per budget
Central coordinator	Preserve shared state and check handoffs	Stale evidence and rejected calls
Learned response selector	Allocate a trial using past feedback	Held-out reward and selection cost

Return to L4 and E5. The learned selector ties majority voting. A post-hoc fixed reactive-power rule produces the same actions. This result makes the added model calls a research question.

Discussion, 30 seconds

If three models propose the same action, what additional information has the ensemble supplied?

Kim et al., agent-system scaling, v3 (2026). Published title: Capable language models can outgrow the benefits of collaboration, Nature Machine Intelligence (2026). Tutorial evidence: E5, saved voltage pilot.

Reusable skills make the procedure explicit

A recent power-system example. Etingov and Biswas combine reusable skills, subagents and MCP-connected PSSE tools for power-flow and dynamic-study tasks.

A reusable procedure needs	Voltage-support example for this classroom
Entry conditions	Current topology, trusted measurements and declared limits
Method and tools	MPC or SAC proposal, model execution and fresh verification
Evidence to return	Candidate ID, model scope, constraint margins and objective
Exit or escalation	Verified recommendation, or a reason to hold for review

Code connection, L1. `bind_evidence` ties the result to its request and candidate. `bound_to` rejects a changed load or command.

A small experiment

Remove the model scope from a handoff. Decide which later claim can no longer be justified.

Etingov and Biswas, Skill-Based AI Agents for Power-System Studies, v2 (1 October 2026), preprint. Context design: Anthropic, Effective context engineering (2025), engineering guidance. Table: proposed classroom procedure.

Control-room authority belongs in executable rules

2026 governance study: an external layer enforces tool permissions, step budgets, operator approval and numerical provenance. Its prototype reports 93.7% task success and 100% rule compliance in 590 runs.



A changed candidate, state or operating limit invalidates the prior review.

The rule can hold

An approval gate blocks unapproved actions.

A number points to a backend result.

The task can still fail

The proposal may name the wrong equipment.

The agent may choose the wrong source.

Lab L12: `operator_packet` checks the required scope. `acknowledge` checks readback identity. The endpoint remains advisory.

Mylonas, Foti and Varvarigos, Governance-Aware Agentic Digital Twin for TSO Control Rooms (18 September 2026), preprint, §6. Diagram: tutorial review boundary.

The next study should isolate workflow value

A testable question: does agentic orchestration complete unfamiliar operator requests more reliably with the same engineering tools and budget?

Comparison	Hold fixed	Measure separately
Scripted workflow vs. Llama orchestration	Models, observations, limits and available tools	Correct intent, completed stages and unsupported claims
One LLM vs. several roles	Base model and total tool/token budget	Handoff errors, recovery and latency
Fixed vs. vote vs. learned selection	Response pool, trial budget and frozen test set	Feasibility, objective and selection overhead

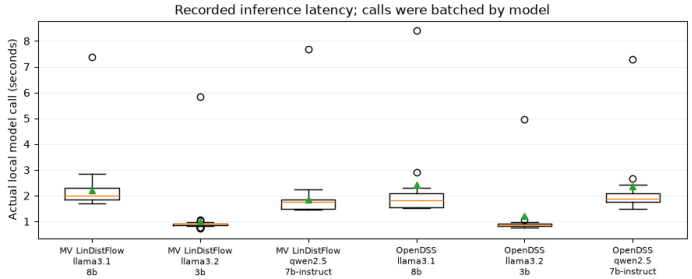
Stress extension: change forecast uncertainty or the model scope, then repeat the audit. Keep test outcomes out of prompts and learned-selector updates.

Exit question

Which measured outcome would justify adding one more agent to your control-room workflow?

Evidence-based scoring: PowerAgentBench-SS (June 2026). Uncertainty-aware orchestration: CONDUCTOR (June 2026). Table: proposed evaluation design, not a reported experiment.

Latency must be measured where the work actually happens



Recorded experiment: 168 real local-model calls. MV mean accounted serial time: learned selection 8.119 s; fixed numerical refinement 3.327 s.

Separate: solver time, LLM load/inference, retries and final verification. Cached notebook display time is not inference latency. The AC transfer reused MPC planning and reports that component separately.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`.

Lab 11: one optional local Llama call, with visible provenance

Notebook L11 Default is offline; explicitly enable the optional local call.

```
client = ProposalLlama(journal, model='llama3.1:8b')
live_reply = client.ask('Optimization Agent', pool['context']['public_context'],
                        list(pool['context']['catalog']))
```

Actual validation, 17 October 2026: Llama 3.1 8B chose `RELAX_Q_10_PERCENT` on the original test_01 context.

Excerpt from the returned public reason

"the RELAX_Q_10_PERCENT operation has the potential to reduce reactive effort magnitude by ten percent, which may improve the cost."

Observed time: 34.113 s client elapsed; Ollama reported 16.967 s loading, 15.192 s prompt processing and 1.916 s generation.

This is one call, shown from its saved record. The default L11 makes no new call. Inspect the complete response and mode in the notebook; a reason still requests a fresh trial.

Ollama chat API; Llama 3 model report (2024); actual record: `results/classroom_optional_llama_validation.json`; Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

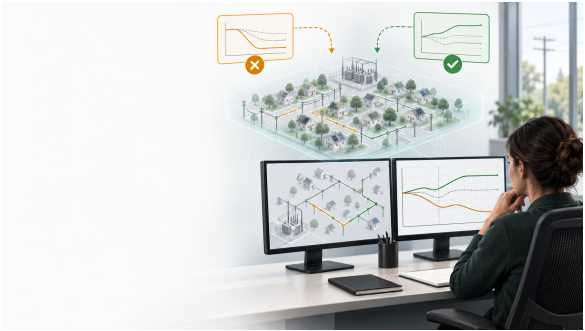
Evaluate the failure, the recovery and the final outcome separately

Metric	Question	Common mistaken conclusion
Task completion	Did the intended study finish?	Valid JSON means success.
Electrical feasibility	Which constraints passed, on which model?	Solver convergence means safe.
Recovery	Was the failed attempt repaired within budget?	More retries mean intelligence.
Claim support	Does each numerical statement match evidence?	A fluent reason is evidence.
Operator benefit	Did correct decisions improve with acceptable workload?	A dashboard proves human benefit.

Our evidence supports an executable teaching workflow. A field claim would need broader scenarios, stronger models and a study with operators.

PFAgent (2026); PowerAgentBench-SS (2026); MAST failure taxonomy.

The control room receives a recommendation with an evidence trail



Concept illustration; AI-generated, not a deployed control room.

What the operator needs

Requested action and exact equipment or candidate ID.

Current case, model scope and constraint margins.

Failed alternatives, unresolved assumptions and age of evidence.

Concise public rationale, with each numerical claim tied to a tool result.

Deployment concept: advisory service beside existing operational applications; authorized procedures govern any later field action.

Governance-aware TSO digital twin (2026); illustration provenance in `figures/Proposal_Illustration_Provenance.md`.

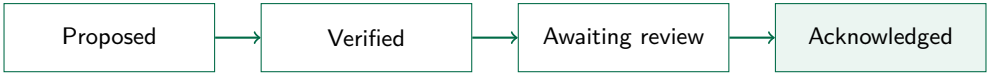
Lab 12: acknowledgment is a state transition, not a generated sentence

Notebook L12 Build an operator packet and test the readback gate.

```
wrong_id = lab.acknowledge(mv_scope, 'wrong-id', 'Student operator')
readback = lab.acknowledge(mv_scope, mv_scope['recommendation_id'], 'Student operator')
```

Your turn: 4 minutes

Try a wrong candidate ID, then the correct readback. Change the requested control after review. Decide whether the old acknowledgment should remain valid.



This is a local advisory exercise. No state in this notebook dispatches equipment.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Runtime authority: Mylonas et al. (2026).

One trace should answer what happened at every stage

Notebook L10 Compare real saved stage records and public responses.

```
archived_reasons = []
for case, backend in [('voltage', 'lindistflow3'), ('restoration', 'lindistflow3'), ('cyber_stealth'
    , 'dc')]:
    run = lab.archived_stage_replies(case, backend)
```

Inspect	What the answer should expose
Input	Case, measurement assumptions, allowed operations and model scope.
Method	Which optimizer, policy, simulator or language call actually ran.
Output	Candidate ID, returned values, constraint failures and public reason.
Cost	Measured tool/call time, including failed attempts.
Disposition	Retained, rejected, refined or escalated; the next required step.

Trace labels matter: classroom fresh solve, archived actual LLM response and optional live call are different evidence modes.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Actual recorded five-stage reference: `Proposal_Complete_Workflow.ipynb`, P1–P8; `results/proposal_complete_workflow.json`.

Capstone: would you approve, hold or request another study?

Card	Evidence presented	Your decision
A	MV PASS; full-feeder AC FAIL; Llama says "ready".	Review / hold / study?
B	Restoration MILP PASS; equipment isolation has no supporting record.	Review / hold / study?
C	No residual alarm; protected measurement set is unavailable.	Review / hold / study?

Task: identify the missing fact and the appropriate tool or human step. Do not answer with a model's confidence score.

Debrief: A requires broader-model repair; B requires authorized isolation evidence; C requires justified independent information or escalation. None supports automatic dispatch.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

Match the architecture to the operator task

Task condition	Useful architecture	Evidence required
Stable repeated calculation	Fixed workflow + numerical checks	Correct model and tested contract.
Varied language requests	LLM tool selection + typed state	Intent accuracy and supported claims.
Several independent studies	Bounded specialist roles	Handoff quality and real parallel benefit.
Competing refinements	Vote or learned selector	Held-out gain at equal trial budget.
High-consequence action	Human review + enforced authorization	Correct action, scope, provenance and permissions.

Design principle: add complexity only when it solves a measured failure of the simpler workflow.

Kim et al., scaling agent systems (2026); governance-aware TSO study (2026).

Explain the full workflow in sixty seconds

Awareness: establish the current case and what is known.

Planning: choose an appropriate numerical method and form candidates.

Simulation: calculate each candidate's consequences in a declared model.

Verification: check physical limits, identity, evidence and permissions.

LLM optimizer: propose a bounded next trial, then return through simulation and verification.

Operator review: inspect the recommendation and unresolved assumptions before any authorized action.

Partner exercise: use one of today's cases and name the actual method and output at every stage.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

Three checks for scientific claims about agentic control

1. Is the comparison matched?

Same observations, model coverage, limits, tools, candidate budget and test scenarios?

2. Which component caused the difference?

Better optimizer, broader model, protected information, language interpretation or coordination?

3. What was actually measured?

Constraint compliance, objective, task completion, claim support, latency or human performance?

Today's result: the framework makes these questions executable. Extra language models and learned selection did not establish general superiority in the recorded pilot.

PowerAgentBench-SS (2026); Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`.

Your next experiment: change one assumption and keep an audit trail

Extension	Change one thing	Keep fixed
Voltage	Forecast uncertainty or a current margin	Controls, model scope and trial budget.
Restoration	Priority weights or one tie rating	Fault, graph and baseline constraints.
Cyber	Sensor trust or noise assumptions	Physical truth and scoring protocol.
Agent design	One role, prompt or selector feature	Tools, test split and numerical gates.

Deliverable: one plot, a baseline comparison, a trace explaining the result and a statement of what remains unproven.

After class: the appendix contains derivations, full code excerpts, recorded responses and training details. It is reference material outside the 180-minute route.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Reading guide: `appendix/Agentic_AI_Research_2026.md`.